

Comparative Analysis of Constituency Parsers: A Case Study on Stanford U-PCFG and Berkeley Neural Parsers

Batu EL

University of Cambridge
be301@cam.ac.uk

Abstract

Constituency parser performance is typically evaluated with bracketing F-score, which is a useful proxy of overall performance but does not adequately reveal the strengths and weaknesses of a system. This report presents quantitative and qualitative evaluations that aim to capture the behavior of a parser under different scenarios through observations of parser performance on a designated test set.¹ To that end, it offers a comparative analysis of two constituency parsers: Berkeley Neural Parser (Kitaev and Klein, 2018; Kitaev et al., 2019) and Stanford Unlexicalized Probabilistic Context-Free Grammar (PCFG) Parser (Klein and Manning, 2003).²

1 Introduction

Historically, parsing—the task of assigning syntactic structure to a sentence—has played a central role in computational linguistics, and it remains important for many NLP applications, such as grammar checking and information extraction. Interpreting linguistic structures is essential for understanding a language and evaluating the performance of systems that process it. Various grammars have been developed to annotate linguistic structures, including phrase structure grammars (Chomsky, 1956; Backus, 1959), dependency grammars (Tesnière, 1959), and categorial grammar formalisms (Bar-Hillel, 1953; Bangalore and Joshi, 1999).

In this paper, I focus on the phrase structure grammar and analyze two systems for constituency parsing: Stanford Unlexicalized Probabilistic Context-Free Grammar (U-PCFG) Parser (Klein and Manning, 2003) and Berkeley Neural Constituency Parser (Kitaev and Klein, 2018; Kitaev et al., 2019). Stanford U-PCFG Parser is based

on an extension of probabilistic context-free grammar (Salomaa, 1969; Booth, 1969), the earliest disambiguation approach for context-free grammar parsing (Jurafsky and Martin, 2023, p. 379), while Berkeley Neural Parser employs a modern self-attentive neural network approach, which underlies the current state-of-the-art systems (Tian et al., 2020).

The paper is structured as follows. Section 2 provides background on constituency parsing before the parsers are described in Sections 3 and 4. In Section 5, I analyze the parsers using evaluation metrics that assess the overall performance. Acknowledging that a more detailed analysis of the parsers' behavior under different scenarios is necessary to understand their strengths and weaknesses (Kummerfeld et al., 2012), I categorize the error types into syntactically meaningful groups in Section 6. For each error category, I make severity assessments, explain how and why the error arises, and suggest methods for addressing the error.

2 Background

Context-free grammar (CFG) (Chomsky, 1956; Backus, 1959)—also called phrase structure grammar and constituency grammar—is a formal system used to model the constituent structure in language.

Definition 1. A context free grammar G is defined by four parameters: N, Σ, R, S , where N is a set of non-terminal symbols, Σ is a set of terminal symbols, R is a set of rules, and S is a designated start symbol. For $r \in R$, $r = \alpha \rightarrow \beta$, where $\alpha \in N$ and $\beta \in (\Sigma \cup N)^*$ (Hopcroft and Ullman, 1979; Jurafsky and Martin, 2023, Chapter 17).

Terminal symbols in Σ represent the words in the language, and the non-terminal symbols in N are abstractions over the terminals. At the first level of abstraction, each terminal is often associated with a Part-of-Speech (POS) tag (Santorini, 1990) from the non-terminals. The rules, R , indicate how the

¹The designated test set is included in Appendix A.

²This report is prepared for the Introduction to Natural Language Syntax and Parsing course at the University of Cambridge.

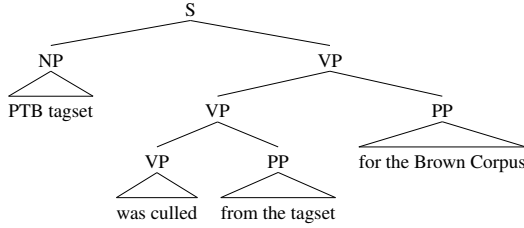


Figure 1: PP Attachment Ambiguity Example. In this interpretation of the sentence, the PP "for the Brown Corpus" is modifying "was culled". This implies that the culling was done for the Brown Corpus. Alternatively, "for the Brown Corpus" can modify "the tagset". In that case, "the tagset for the Brown Corpus" would be treated as a single unit, constituting an NP to which the PP is attached.

symbols in N and Σ can be combined, defining a set of possible strings in the language. This formalism can be used to generate new sentences using the rules and symbols in the grammar, or it can be used to assign structure to sentences, i.e., parsing. The latter is the focus of this paper.

Cocke-Kasami-Younger (CKY) Algorithm (Kasami, 1965; Younger, 1967) is a dynamic programming approach for syntactic parsing. An augmented CKY algorithm that stores pointers to keep track of derivation paths and allows multiple versions of the same non-terminals can be used to efficiently compute the set of possible parse trees for a given sentence (Jurafsky and Martin, 2023, p. 370).³ However, there are often multiple possible parses of a sentence due to syntactical ambiguities, i.e., a constituent can be attached to different places in the parse tree (See Figure 1), and the CKY algorithm is not equipped to identify the correct parse from this set of possible parses. Therefore, parsers tackle the problem of ambiguity by assigning a score (or probability) to each valid parse tree and selecting the tree with the highest score as the best parse.

This paper focuses on two parsers that employ different scoring functions aimed at the problem of syntactic disambiguation. At inference time, both parsers use variants of the CKY algorithm to generate a set of valid parse trees for the given

³CKY requires the grammar to be in Chomsky Normal Form (Chomsky, 1963), a standard form for context-free grammars where the rules with non-terminal right-hand sides have to be binary, unary rules expand to single terminals, and none of the production rules results in an empty symbol. Notably, this does not restrict the applicability of CKY as any CFG can be converted into a weakly equivalent Chomsky normal form grammar (Jurafsky and Martin, 2023, Chapter 17).

	Stanford U-PCFG	Berkeley Neural
Parameterization	CFG Rules	Constituent Spans
Estimation	MLE	Neural Network
Aggregation	Product	Sum

Table 1: Differences in Tree Score Calculation.

sentence. Both calculate scores for each of the trees in this set and identify the correct parse as the valid tree with the highest score. However, the parsers differ in the way that they calculate the scores for the valid parse trees, which is reflected in the differences in their outputs.

Stanford U-PCFG and Berkeley Neural parsers employ tree scoring functions that differ in three major dimensions. Table 1 summarizes these differences. Firstly, the parsers use different parameters based on which they compute the tree scores. In Stanford U-PCFG Parser, the tree score calculation problem is parameterized in terms of the context-free rules, whereas the Berkeley Neural Parser uses span-constituent pairs to parameterize its tree scoring function. Secondly, while the Stanford U-PCFG parser uses maximum likelihood estimation to calculate the probability for rule expansions, the Berkeley Neural Parser employs neural encoder-decoder architecture to map sentences to the scores for span-constituent pairs. Thirdly, the Stanford U-PCFG parser calculates the score for a parse tree by multiplying the probabilities for each grammar rule used in the tree. On the other hand, Berkeley Neural Parser calculates the score for a parse tree by summing the scores for span-constituent pairs for each of the constituents in the parse tree. The next sections describe the parsers in detail.

3 Stanford U-PCFG Parser

Stanford U-PCFG Parser⁴ (Klein and Manning, 2003) is based on probabilistic context-free grammar (PCFG) (Salomaa, 1969; Booth, 1969), which is the earliest disambiguation approach for CFG parsing (Jurafsky and Martin, 2023, p. 379). PCFG extends CFG by augmenting each rule $r \in R$ from Definition 1 with a probability $P(\alpha \rightarrow \beta)$,

⁴For my experiments, I used the "englishPCFG" model from the "lexparser" folder of version 4.2 of the Stanford Parser with the default settings via nltk integration. This model is trained on the sections 2-21 of the WSJ section of the Penn treebank (Marcus et al., 1993; Taylor et al., 2003)

such that $\sum_{\beta \in (\Sigma \cup N)^*} P(\alpha \rightarrow \beta) = 1$.

The rules are assumed to be conditionally independent; hence, the probability (score) of a parse tree composed of n rules is calculated as the product of the probabilities for the rules present in the parse:

$$P(T) = \prod_{i=1}^n P(\alpha_i \rightarrow \beta_i) \quad (1)$$

where α_i (β_i) represents the left (right) hand side and of the i^{th} rule used in the tree.

Stanford U-PCFG parser uses unsmoothed maximum likelihood estimation to calculate the probability for the non-terminal⁵ rule expansions.

$$P(\alpha \rightarrow \beta) = \frac{\text{Count}(\alpha \rightarrow \beta)}{\text{Count}(\alpha)} \quad (2)$$

3.1 Markovization

The choice of representation can significantly impact PCFG-based parsers' performance (Collins, 2003; Johnson, 1998). To reduce the biases in PCFG that arise from the independence assumption in Equation 1, Stanford U-PCFG parser employs *parent annotation*⁶ by modifying the labels of the nodes in the parse tree so that they contain information about their parents. The richer node labels result in a grammar with many context-free rules. However, instances of certain rules become extremely rare in the training data, which increases the variance of probability estimates (Klein and Manning, 2003; Johnson, 1998). Stanford U-PCFG parser addresses this sparsity issue by horizontal Markovization, i.e., Markovization of the rules (Collins, 1999).⁷

3.2 Other Annotations

The absence of lexical information in unlexicalized PCFG results in difficulties in resolving ambiguities (Klein and Manning, 2003). To that end, the Stanford U-PCFG parser splits pre-terminals.⁸ Moreover, it uses a series of other annotations that have been observed to benefit the parser's F_1 -score performance, including unary annotations⁹, subcategorization of function words¹⁰, and others¹¹.

⁵The calculation for the POS tag probabilities is explained in Appendix B.1

⁶Further discussion of vertical Markovization via parent annotation is included in Appendix B.2

⁷Further discussion of horizontal Markovization of the rules is included in Appendix B.3

⁸A discussion of tag splitting is included in Appendix B.5

⁹Unary annotations are discussed in Appendix B.4

¹⁰See Appendix B.6

¹¹See Appendix B.7

3.3 Inference

At inference time, the Stanford U-PCFG parser uses an array-based Java implementation of CKY (Klein and Manning, 2003) to parse this grammar and generate a set of valid parse trees for the given sentence. The score for each valid parse tree is calculated based on the probabilities estimated at training time using Equation 1, and the valid tree with the highest score is selected.

4 Berkeley Neural Parser

Berkeley Neural Parser¹² (Kitaev and Klein, 2018; Kitaev et al., 2019) is a neural constituency parser with an encoder-decoder architecture. The parser's scoring function represents a constituency tree T with $|T|$ constituents as a set of triples (i_t, j_t, l_t) for $t = 1, \dots, |T|$, where the t^{th} constituent is located between *fencepost* positions i_t and j_t and has label l_t (Kitaev et al., 2019).

$$T = \{(i_t, j_t, l_t) : t = 1, \dots, |T|\}$$

The parser assigns a score to each parse tree using the scoring function

$$s(T) = \sum_{(i,j,l) \in T} s(i, j, l) \quad (3)$$

4.1 Encoder

To estimate the scores $s(i, j, l)$, firstly, the Berkeley Neural parser uses two self-attentive encoders (Vaswani et al., 2017) to build vector representations for each position that contain information about the relationships between the words in the sentence.

The first set of encoder layers uses a pre-trained BERT (Devlin et al., 2019) encoder to map words to vectors.¹³ The second set of encoder layers takes as input the output of the first encoder, positional embeddings, and word vectors from a word-based tokenization (Kitaev and Klein, 2018). These vectors are passed through 8 identical transformer layers with block sparsity constraints¹⁴ (Kitaev and Klein, 2018; Kitaev et al., 2019). The output of the

¹²For my experiments, I used the "benepar_en3_large" model with the default settings. I accessed the model via the "en_core_web_md" pipeline from the SpaCy library. The model is trained on the revised WSJ training set from Bies et al. (2015)

¹³Since BERT maps sub-word units based on WordPiece tokenization (Wu et al., 2016) to vectors, only the last sub-word unit for each word is used as the word vector (Kitaev et al., 2019).

¹⁴Block sparsity constraints are discussed in Appendix C.1

encoder blocks, y , contains vectors $[y_1, y_2, \dots, y_T]$ corresponding to each position in the sentence.

4.2 Decoder

Using y , the system then constructs vectors $v_{i,j}$ to represent each span (i, j) by subtracting the representations learned for the positions at the beginning and end of the span (Kitaev and Klein, 2018).¹⁵

The decoder maps the vector $v_{i,j}$ to $s(i, j, \cdot)$ using a feed-forward classifier with two fully connected layers and ReLU activation.

$$s(i, j, l) = M_2 \text{ReLU}((M_1 v_{i,j} + c_1)) + c_2 \quad (4)$$

where M_i are learned matrices. The score $s(i, j, l)$ represents the likelihood of an l constituent between fencepost positions i and j .¹⁶

4.3 Inference

At inference time, Berkeley Neural parser uses a chart-based Cython implementation of CKY¹⁷ recursion (Stern et al., 2017; Kitaev and Klein, 2018) to generate a set of valid parse trees for the given sentence. The score for each valid parse tree is calculated using Equation 3 with scores from Equation 4, and the valid tree with the highest score is selected.

$$\hat{T} = \arg \max_T s(T)$$

5 Evaluation

5.1 Runtime

Figure 2 shows the runtime of the parsers for different sentence lengths. Stanford U-PCFG parser is two times slower than the Berkeley Neural Parser for shorter sentences. Moreover, the Stanford U-PCFG parser is significantly slower for longer sentences, while the Berkeley Neural Parser’s runtime is not significantly affected by sentence length. Notably, the runtime remains below 0.5 seconds for all sentences in the designated test set.

The worst-case runtime of both the chart-based (Berkeley Neural Parser) and the array-based (Stanford U-PCFG Parser) implementations of CKY is $O(n^3)$, where n is the length of the sentence

¹⁵The details are discussed in Appendix C.2

¹⁶The model is trained by minimizing hinge loss. A detailed discussion of training procedures can be found in (Kitaev and Klein, 2018)

¹⁷Conversion to Chomsky Normal form is discussed in Appendix C.3

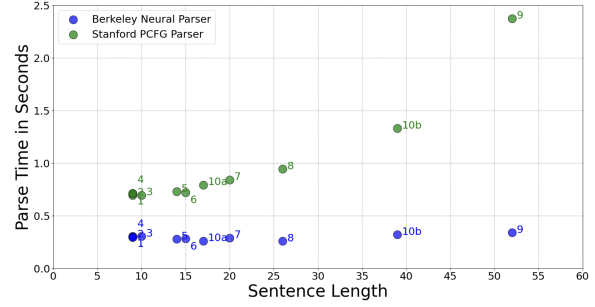


Figure 2: Parsing Time vs. Sentence Length. The reported numbers are the average of 100 experiments. Sentence lengths are calculated using EVALB.

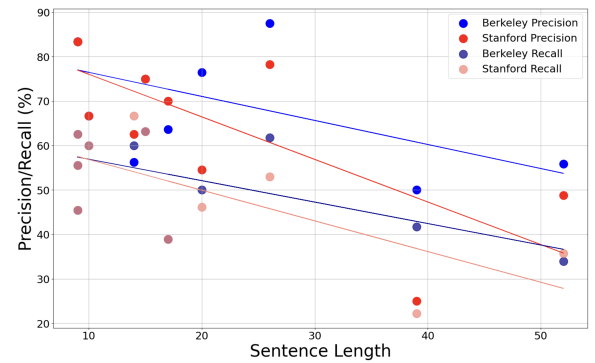


Figure 3: Precision and Recall vs. Sentence Length.

(Hopcroft and Ullman, 1979, p. 140). In Berkeley Neural Parser, CKY consumes a negligible fraction of the runtime, which is dominated by the neural network computations (Kitaev and Klein, 2018); therefore, the polynomial trend in runtime is not observed. In contrast, the annotation steps in the Stanford U-PCFG parser increase the number of rules in the grammar. When parsing this grammar with CKY, longer runtimes are observed for all input sizes. Since the CKY algorithm dominates the runtime of this parser, a polynomial trend is observed.

5.2 PARSEVAL Metrics

Figure 3 shows the labeled precision and recall with respect to the sentence length. The statistics are calculated using the evalb implementation (Sekine and Collins) of the PARSEVAL (Black et al., 1991) metrics. Appendix D describes the preprocessing steps followed to prepare the parser outputs and the provided gold standard parses for evaluation in this and the following sections.

On the designated test set, Berkeley Neural Parser achieves an overall Bracketing F_1 -score of

56.12¹⁸, which is significantly lower than the reported 96.29 F_1 -score on the revised WSJ test set (Kitaev, 2024).

Similarly, the Stanford U-PCFG parser experiences a significant drop in performance on the designated test set with an overall bracketing F_1 -score of 50.35. The parser is reported to achieve 86.36% F_1 -score on WSJ section 23 from the Penn treebank (Klein and Manning, 2003).

In general, recall is relatively lower compared to precision for both parsers. This indicates that the syntactical structures identified by the parsers often match the gold standard, but the systems fail to capture some structures in the gold standard. A careful inspection of the parser outputs reveals that this is likely because the gold standard trees more often prefer binary branching, while Berkeley and Stanford parser outputs frequently select trees with higher-order branching. For Stanford U-PCFG parser, the reasoning behind this phenomenon can be traced back to the scoring function: Since the score for a tree is calculated as the product of probabilities (Equation 1), where each probability is between 0 and 1 (Equation 2), having more rules would decrease the tree score in a hypothetical case where all rules have equal probability.

The parsers have similar precision and recall for shorter sentences. However, with increasing sentence length, the decline in precision and recall is steeper for the Stanford Parser, which results in the differences observed in overall performance. A similar trend is observed with the crossing brackets (see Tables 5 and 6). Moreover, I noted that the decline in precision with sentence length is steeper in the Stanford parser. The impact of the multiplicative nature of the scoring function seems to be amplified for larger trees with more nodes.

Notably, the outputs of the two parsers match exactly for 5 of the 11 sentences (1, 2, 3, 4, and 6). This indicates that while the parser outputs often differed from the gold standard, the parsers were aligned with each other, making similar mistakes.¹⁹ I also noted that the parser outputs were more aligned for shorter sentences.²⁰

5.3 POS Tags

We note that 33 unique POS tags have been used in the gold standard parses of 11 sentences. Similarly, the Berkeley parser used 33 distinct tags.

Tag Set	Tags
Gold Standard \Stanford	JJR, RBS
Gold Standard \Berkeley	JJR
Stanford \Gold Standard	RBR, NNPS, JJS FW, SYM
Berkeley \Gold Standard	RBR

Table 2: Comparison of POS Tags Sets of Stanford and Berkeley Parsers with the Gold Standard. X \Y indicates the set of POS tags used by X but have not been used by Y. Yellow highlights indicate the tags in the Gold Standard that have not been used by either of the parsers. Orange highlights represent the tags used by both parsers that have not been used in the Gold Standard.

Meanwhile, the Stanford parser used all 36 tags identified by the Penn Treebank Project (Santorini, 1990; Marcus et al., 1993). Table 2 elaborates on these differences.

The confusion matrices in Appendix K reveal that both parsers confuse JJR with RBR in both of the two occurrences of the JJR tag, which explains the trend in Table 2. These occurrences are in sentence 9 in the expression "more or less". The Penn Treebank guidelines verify that the gold standard tagging is correct, stating that the correct tag in the expression "more or less" is JJR (Santorini, 1990). However, in many other expressions, "more" and "less" are often tagged as comparative adverbs (RBR)²¹. RBR tagging is likely to be more frequent in the training data, undermining the accuracy of the taggers when faced with "more or less".

6 Categorization of Errors

While the metrics discussed in Section 5.2 are useful to measure the overall performance of the parser, a more detailed analysis of the parsers' behavior under different scenarios is necessary to understand the parsers' strengths and weaknesses (Kummerfeld et al., 2012). In this section, I classify the errors into linguistically meaningful types using the error categorization method proposed by Kummerfeld et al. (2012).

This algorithm Kummerfeld et al. (2012, Algorithm 1) categorizes the node errors using tree transformations to repair the groups of errors. Firstly,

¹⁸The full table is included in the Appendix E, see Table 5.

¹⁹see Table 7

²⁰The full table is included in the Appendix E, see Table 7.

²¹"I can't go on like this any/RB more/RBR." (Santorini, 1990)

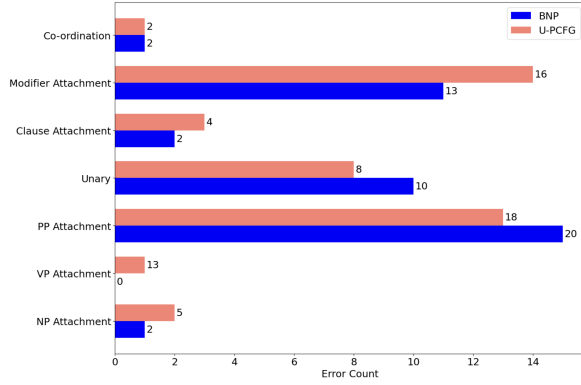


Figure 4: 7 Error Types from Kummerfeld et al. (2012). The bars represent the number of occurrences of the error types, and the numbers on the right are the bracket errors associated with the occurrences of the error type.

it identifies a set of tree transformations that can transform the parser output to the gold tree. Then, the transformations that fit pre-defined error templates are classified into error types. When the algorithm identifies a suitable template for an error, it corrects the error by moving the subtrees (Kummerfeld et al., 2012). Finally, the templates used for correcting tree structures are reported when the algorithm can no longer identify any pre-defined errors to correct in the parse tree.

6.1 VP Attachment

A single attachment error can lead to many node errors and significantly impact the bracketing scores of the system overall. The single VP attachment error that leads to 13 bracket errors in the Stanford Parser’s output for sentence 10b²² demonstrates this phenomenon (see Figure 4 and Appendix F), highlighting the inadequacy of the summary statistics discussed in Section 5.2. This error also explains the particularly low labeled precision and recall observed in Table 6.²³ This is the only VP attachment error encountered in the designated test set.

An inspection of the parser output and the gold standard shows that the error occurs due to the difficulty of identifying the linguistic structure of a sentence that talks about language. In the sentence, the verbs that are listed as examples ("do", "doing", "be", and "have") are treated as individual verbs by

²²10b: For example the original Brown and C5 tagsets include a separate tag for each of the different forms of the verbs **do** (e.g. C5 tag VDD for did and VDG tag for **doing**), **be** and **have**.

²³LR = 22.22 and LP = 25.00 for sentence 10b

the Stanford parser, and each is assigned a separate verb phrase. However, these words do not play a verbal role in the context of this sentence. They are rather constructions that are used to refer to the verbs, i.e., names of the verbs. Accordingly, they are treated simply as list elements grouped under LST in the gold standard—though they are still POS-tagged as verbs.

This distinction is especially difficult for an unlexicalized parser, which has to predict which token labeled as a verb acts as a verb in the sentence without knowing the word attached to the POS tag. Meanwhile, I observed that the Berkeley Neural parser, which has lexical information via the word embeddings, recognizes that these verbs do not form separate VPs and groups these items under a UCP (Unlike Coordinated Phrase) constituent, which marks elements that belong to different syntactic categories grouped together (Prolo, 2006).

VP attachment error is a severe error that misinterprets the core structure of the sentence. Vulnerability against this error type undermines Stanford Parser’s usability, especially for texts about linguistic structures. Nevertheless, the Stanford parser has been observed to benefit from domain adaptation for specific contexts (Wang et al., 2015). This approach can be used to mitigate VP attachment errors when parsing texts on language.

6.2 PP Attachment

PP attachment errors are instances when PP attachment ambiguities (Figure 1) are not resolved correctly by the parser. Appendix G contains an example from the Berkeley Parser’s output for sentence 6²⁴, where the PP "at auction" is attached at the same level as "last year" and "in February". In contrast, in the gold standard, the PP "at auction" is treated to be more closely related to the verb "sold", forming the VP unit "sold at auction". PP attachment errors can be detrimental for syntax-based machine translation systems, where the whole translation can be considered unlikely if a PP attachment is not likely (Sumita et al., 1993). I observed high rates of PP attachment errors from both parsers. This points to the difficulty in resolving PP attachment ambiguities and can obscure the success of machine translation systems that use one of the parsers as an intermediary step.

²⁴6: It was my aunt’s car which we sold at auction last year in February.

6.3 NP Attachment

NP attachment errors are relatively rare, but Appendix H contains an example that is worth discussing, as both parsers struggle to parse this short phrase ($F_1 \approx 55$). In sentence 5²⁵, the subject needs to be singular to agree with the singular verb "has". Based on this insight, we know that in the valid parse of the sentence "as well as the rabbits which we wanted to eat" needs to constitute a single unit, which is labeled as CONJP in the gold standard.

Berkeley parser fails to capture this nuance by grouping "the horse as well as the rabbits" into a single NP constituent, forming a plural subject. Meanwhile, the Stanford parser correctly identifies "The horse" and "the rabbits which we wanted to eat" as two separate NPs. However, the parser puts these two NPs connected by a CONJP at the same level, treating them as two equal contributors of the NP at the higher level, which would still require a plural verb for the sentence to be grammatically correct.

The correct attachment of noun phrases is important for determining the meaning of a sentence. Overall, NP Attachment errors are more common in the Stanford parser's outputs. This undermines Stanford Parser's usability in question-answering and information extraction applications, as well as other tasks that require extracting structured information.

6.4 Modifier Attachment

The cases that involve misplaced adverbs and adjectives are labeled as modifier attachment errors. Appendix I includes an example from the output of the Stanford parser for sentence 4²⁶. In this example, the Stanford parser attaches the adverb "both" too high in the tree.

Notably, both parsers tag "both" as a determiner at the POS level, while the gold standard tag treats it as an adverb. There are no clear guidelines in the Penn Treebank to make this distinction (Santorini, 1990), although "both" is mentioned as a determiner, a predeterminer, and a conjunction in different contexts. Nevertheless, in English, determiners come before the nouns that they modify (cam), and "both" is followed by a verb in this sentence. While the usage of "both" as a determiner is

common (ex. both books are blue), in this sentence, "both" modifies the verb "broke up", hence should be labeled as an adverb.

Modifier errors are more common in the output of the Stanford parser. This error type is arguably less critical compared to the previous ones, as modifiers provide additional information that is not central to the core meaning of the sentence.

6.5 Clause Attachment

Appendix J has an example from sentence 8²⁷, where an S node is misplaced. The clause attachment structure in the sentence is influenced by the tag of "so". Both parsers identify two sentences, labeled S, joined by "so", labeled as a subordinating conjunction. On the other hand, in the gold standard, the sentence is divided into three components: NP, VP, and S, where "so" is labeled as an adverb.

Santorini (1990) provides clear guidelines for tagging "so", stating that when it means "so that" it should be tagged as a subordinating conjunction, and when it is used in the sense of "therefore" it should be tagged as adverb (Santorini, 1990, p. 29, 30). In this example, "so" is used in the sense of "therefore". Hence, it should be tagged as RB according to the guidelines.

It is worth noting that this is a particularly challenging tagging task for the Stanford Parser, which does not have the machinery to accurately capture the sentence meaning, since the label of "so" depends on the meaning of the whole sentence. While the Berkeley Neural Parser has the machinery to capture sentence meaning via word embeddings and transformer architecture, it still fails to identify the correct tag. Clause attachment errors can be significant in downstream tasks that involve legal documents, where incorrect attachment of clauses can alter the interpretation of terms and conditions.

6.6 Unary

Figures 5 and 6 demonstrate an example of a unary error resulting from missing unary production. The gold standard contains two unary productions (NP $\rightarrow$ NNP), which are not present in the Stanford parser's output (Figure 6). This error type is observed more often in the Berkeley parser's outputs, although it is less significant than the other errors because missing unary nodes do not alter the over-

²⁵5: *The horse as well as the rabbits which we wanted to eat has escaped.*

²⁶4: *Kim and Sandy both broke up with their partners.*

²⁷8: *Letters delivered on time by old-fashioned means are increasingly rare, so it is as well that that is not the only option available.*

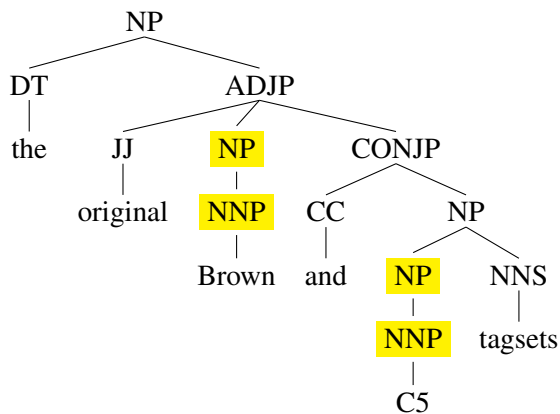


Figure 5: Gold Tree for Sentence 10b

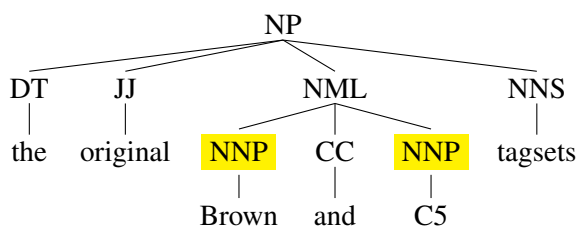


Figure 6: Stanford Parser Output for Sentence 10b

all structure of the parse tree or the semantic interpretation of the sentence.

6.7 Coordination

The only instance of a coordination error in the test set is demonstrated in Figures 5 and 6. In this example, the immediate siblings of the conjunction "and" are misplaced. The conjunction "and" connects "Brown" and "C5 tagsets" in the gold standard, while it joins "Brown" and "C5" in the Stanford parser's output. This is an example of an instance where the Stanford parser prefers higher-order branching, likely due to the multiplicative nature of its scoring function.

Coordination errors are significant since accurate interpretation of the relationships between different parts of a sentence is essential to sentence meaning. Misinterpretations of these relationships can lead to misunderstandings, which can compromise the reliability of the systems when analyzing medical, scientific, or legal texts.

6.8 Attachment Level

Correct attachment level is difficult to capture with the PCFG framework because high and low attachments of a node can result in trees that have similar rules (Klein and Manning, 2003). In those cases, it is difficult to identify the correct parse with a system parameterized by the rules. This situation

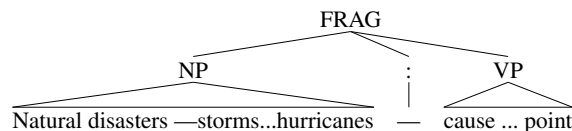


Figure 7: Attachment Error for "—" in Stanford Parser's output for Sentence 7²⁸

can be observed in Figure 7, where the Stanford parser attached "—" directly to the root of the tree, which is labeled as FRAG. In contrast, the Berkeley parser, which is parameterized in terms of spans, correctly identifies that the span of "—" is unlikely to be the whole sentence. The observed performance difference demonstrates one of the strengths of the span-based tree-scoring approach. This is a significant error as it completely alters the core structure of the parse tree and the meaning of the sentence.

7 Conclusion

To conclude, I found that both parsers performed significantly worse on the designated test set than their reported results on the WSJ section 23. While the parser outputs often differed from the gold standard, they were aligned with each other, perfectly matching in 5 out of 11 sentences, suggesting that the two parsers often make similar mistakes.²⁹

When their outputs differed, Berkeley Neural Parser was more aligned with the gold standard, outperforming Stanford U-PCFG parser in most evaluations. It had lower bracketing errors and less frequently made VP, NP, clause, and modifier attachment errors. Meanwhile, the Stanford parser performed better with unary and PP attachment errors. Based on the error rates of the parsers, modifier attachment and PP attachment errors appeared to be the most challenging error types.

The modern neural approach of the Berkeley parser seems to have addressed some of the challenges of the older PCFG method, though this comes at the cost of interpretability. The Berkeley parser's approach effectively handles data from a diverse range of contexts. In contrast, the Stanford parser can require domain adaptations for specific contexts, such as texts that are about language. The choice between the two parsers highly depends on the specific use case and the text that will be parsed. If the text contains long sentences, the Berkeley parser can be preferable since its runtime remains relatively stable across different sentence lengths.

²⁹see Table 7

8 Limitations

The error classification system from Kummerfeld et al. (2012)³⁰, which is used to categorize the parser errors in this report, has limitations. Firstly, the error classification templates were manually constructed by the authors, who defined errors in terms of extra and missing spans, but these definitions are open to questioning. Secondly, the greedy bottom-up implementation in Kummerfeld et al. (2012, Algorithm 1) does not necessarily find the shortest path between the parser output and the gold tree, which can lead to misclassification of certain errors. While Kummerfeld et al. (2012) argues that their experiments with more accurate algorithms yielded similar results with their approach, the coordination error present in the Berkeley parser’s output for sentence 10b was not detected by their system. This is manually corrected in the reported results.

While the Berkeley parser outperforms the Stanford U-PCFG parser in most evaluations, part of the performance difference can be due to its training on the revised Penn Treebank, where the errors from the older version are corrected. In addition to the information in the Penn Treebank training set, the Berkeley parser also uses information from the encoder pre-trained on other datasets, which arguably makes the comparison unfair.

Other evaluation methods that are outside the scope of this paper can reveal different strengths and weaknesses of the two parsers. For instance, Carroll et al. (1998) suggested a dependency relationship extraction-based scheme for parser evaluation. The literature on dependency extraction (King et al., 2003; Briscoe and Carroll, 2006; LIN, 1998) can inform such evaluation methods.

Based on my analysis of the alignment of the crowd-sourced gold-standard with the Penn Treebank guidelines, I concluded that the gold trees follow annotation standards. However, a more elaborate check is necessary to ensure their consistency. In some cases, I found that the guidelines do not provide clear guidance on the correct procedures, which limits the objectivity of the evaluations, leaving room for ambiguities.

Finally, a larger test set is necessary to ensure the robustness of the findings. Nevertheless, the severity of the discussed error types can vary for different downstream tasks. Ideally, an extrinsic

evaluation of parsers should inform whether the parsing system is appropriate for the specific purpose.

References

- Determiner - definition.** Cambridge University Press.
- John Warner Backus. 1959. **The syntax and semantics of the proposed international algebraic language of the zurich acm-gamm conference.** In *IFIP Congress*.
- Srinivas Bangalore and Aravind K. Joshi. 1999. **Supertagging: An approach to almost parsing.** *Computational Linguistics*, 25(2):237–265.
- Yehoshua Bar-Hillel. 1953. **A quasi-arithmetical notation for syntactic description.** *Language*, 29(1):47–58.
- Ann Bies, Justin Mott, and Colin Warner. 2015. **English news text treebank: Penn treebank revised.** Web Download.
- E. Black, S. Abney, D. Flickenger, C. Gdaniec, R. Grishman, P. Harrison, D. Hindle, R. Ingria, F. Jelinek, J. Klavans, M. Liberman, M. Marcus, S. Roukos, B. Santorini, and T. Strzalkowski. 1991. **A procedure for quantitatively comparing the syntactic coverage of English grammars.** In *Speech and Natural Language: Proceedings of a Workshop Held at Pacific Grove, California, February 19-22, 1991*.
- Taylor L. Booth. 1969. **Probabilistic representation of formal languages.** In *Scandinavian Workshop on Algorithm Theory*.
- Ted Briscoe and John Carroll. 2006. **Evaluating the accuracy of an unlexicalized statistical parser on the PARC DepBank.** In *Proceedings of the COLING/ACL 2006 Main Conference Poster Sessions*, pages 41–48, Sydney, Australia. Association for Computational Linguistics.
- John Carroll, Ted Briscoe, and Antonio Sanfilippo. 1998. **Parser evaluation: a survey and a new proposal.** In *International Conference on Language Resources and Evaluation*.
- N. Chomsky. 1956. **Three models for the description of language.** *IRE Transactions on Information Theory*, 2(3):113–124.
- N. Chomsky. 1963. Formal properties of language. In D. Luce, editor, *Handbook of Mathematical Psychology*, volume 2. John Wiley & Sons.
- Michael Collins. 1999. Head-driven statistical models for natural language parsing. *Ph.D. Thesis*.
- Michael Collins. 2003. **Head-driven statistical models for natural language parsing.** *Computational Linguistics*, 29(4):589–637.

³⁰<https://github.com/jkkummerfeld/berkeley-parser-analyser>

- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. [BERT: Pre-training of deep bidirectional transformers for language understanding](#). In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.
- W. Nelson Francis. 1965. [A standard corpus of edited present-day american english](#). *College English*, 26(4):267–273.
- John E. Hopcroft and Jeffrey D. Ullman. 1979. Introduction to automata theory, languages and computation.
- Mark Johnson. 1998. [PCFG models of linguistic tree representations](#). *Computational Linguistics*, 24(4):613–632.
- Dan Jurafsky and James H. Martin. 2023. *Speech and Language Processing (3rd ed. draft)*.
- Tadao Kasami. 1965. [An efficient recognition and syntax-analysis algorithm for context-free languages](#).
- Tracy Holloway King, Richard Crouch, Stefan Riezler, Mary Dalrymple, and Ronald M. Kaplan. 2003. [The PARC 700 dependency bank](#). In *Proceedings of 4th International Workshop on Linguistically Interpreted Corpora (LINC-03) at EACL 2003*.
- Nikita Kitaev. 2024. Self-attentive parser: High-accuracy nlp parser with models for 11 languages. <https://github.com/nikitakit/self-attentive-parser>.
- Nikita Kitaev, Steven Cao, and Dan Klein. 2019. [Multilingual constituency parsing with self-attention and pre-training](#). In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 3499–3505, Florence, Italy. Association for Computational Linguistics.
- Nikita Kitaev and Dan Klein. 2018. [Constituency parsing with a self-attentive encoder](#). In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2676–2686, Melbourne, Australia. Association for Computational Linguistics.
- Dan Klein and Christopher D. Manning. 2003. [Accurate unlexicalized parsing](#). In *Proceedings of the 41st Annual Meeting of the Association for Computational Linguistics*, pages 423–430, Sapporo, Japan. Association for Computational Linguistics.
- Jonathan K. Kummerfeld, David Hall, James R. Curran, and Dan Klein. 2012. [Parser showdown at the Wall Street corral: An empirical investigation of error types in parser output](#). In *Proceedings of the 2012 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning*, pages 1048–1059, Jeju Island, Korea. Association for Computational Linguistics.
- DEKANG LIN. 1998. [A dependency-based method for evaluating broad-coverage parsers](#). *Natural Language Engineering*, 4(2):97–114.
- Mitchell P. Marcus, Beatrice Santorini, and Mary Ann Marcinkiewicz. 1993. [Building a large annotated corpus of English: The Penn Treebank](#). *Computational Linguistics*, 19(2):313–330.
- Carlos A. Prolo. 2006. [Handling unlike coordinated phrases in TAG by mixing syntactic category and grammatical function](#). In *Proceedings of the Eighth International Workshop on Tree Adjoining Grammar and Related Formalisms*, pages 137–140, Sydney, Australia. Association for Computational Linguistics.
- Paul Rayson and Roger Garside. 1998. The claws web tagger.
- Arto Salomaa. 1969. [Probabilistic and weighted grammars](#). *Information and Control*, 15(6):529–544.
- Beatrice Santorini. 1990. Part-of-speech tagging guidelines for the penn treebank project (3rd revision).
- Satoshi Sekine and Michael John Collins. Evalb. <https://nlp.cs.nyu.edu/evalb/>.
- Mitchell Stern, Jacob Andreas, and Dan Klein. 2017. [A minimal span-based neural constituency parser](#). In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 818–827, Vancouver, Canada. Association for Computational Linguistics.
- E. Sumita, O. Furuse, and H. Iida. 1993. An example-based disambiguation of prepositional phrase attachment. *IEEE Transactions on Medical Imaging*.
- Ann Taylor, Mitchell Marcus, and Beatrice Santorini. 2003. [The penn treebank: An overview](#).
- L. Tesnière. 1959. *Éléments de syntaxe structurale*. Éditions Klincksieck.
- Yuanhe Tian, Yan Song, Fei Xia, and Tong Zhang. 2020. [Improving constituency parsing with span attention](#). In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1691–1703, Online. Association for Computational Linguistics.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. [Attention is all you need](#). In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc.
- Yan Wang, Serguei V. S. Pakhomov, James O. Ryan, and G. Melton. 2015. [Domain adaption of parsing for operative notes](#). *Journal of biomedical informatics*, 54:1–9.
- Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V. Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, Jeff Klingner, Apurva Shah, Melvin Johnson, Xiaobing

Liu, Łukasz Kaiser, Stephan Gouws, Yoshikiyo Kato, Taku Kudo, Hideto Kazawa, Keith Stevens, George Kurian, Nishant Patil, Wei Wang, Cliff Young, Jason Smith, Jason Riesa, Alex Rudnick, Oriol Vinyals, Greg Corrado, Macduff Hughes, and Jeffrey Dean. 2016. [Google’s neural machine translation system: Bridging the gap between human and machine translation](#).

Daniel H. Younger. 1967. [Recognition and parsing of context-free languages in time n³](#). *Information and Control*, 10(2):189–208.

A Designated Test Set

#	Designated Set of Sentences
1	My aunt’s can opener can open a drum.
2	The old car broke down in the car park.
3	At least two men broke in and stole my TV.
4	Kim and Sandy both broke up with their partners.
5	The horse as well as the rabbits which we wanted to eat has escaped.
6	It was my aunt’s car which we sold at auction last year in February.
7	Natural disasters – storms, flooding, hurricanes – occur infrequently but cause devastation that strains resources to breaking point.
8	Letters delivered on time by old-fashioned means are increasingly rare, so it is as well that that is not the only option available.
9	English also has many words of more or less unique function, including interjections (oh, ah), negatives (no, not), politeness markers (please, thank you), and the existential ‘there’ (there are horses but not unicorns) among others.
10a	The Penn Treebank tagset was culled from the original 87-tag tagset for the Brown Corpus.
10b	For example the original Brown and C5 tagsets include a separate tag for each of the different forms of the verbs do (e.g. C5 tag VDD for did and VDG tag for doing), be and have.

Table 3: Designated Test Set

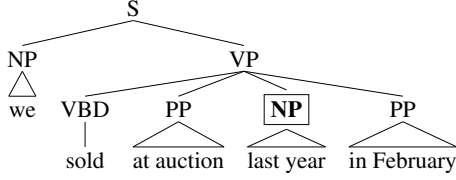


Figure 8: Sentence 6. Stanford Parser Output. Note that this is not the gold standard; hence, the parse here does not represent the correct parse of the sentence.

B Stanford U-PCFG Parser Details

B.1 POS Tag Probabilities

Stanford U-PCFG parser uses smoothed maximum-likelihood estimation to calculate $P(\text{tag}|\text{word})$ at training time (Klein and Manning, 2003). Firstly, the words are categorized into *wordclasses* based on their features like capitalization, suffix, digit, and others. The probability $P(\text{tag}|\text{wordclass})$ is calculated for each category, which is then used as a prior to calculate $P(\text{tag}|\text{word})$:

$$P(\text{tag}|\text{word}) = \frac{\text{count}(\text{tag}, \text{word}) + \kappa P(\text{tag}|\text{wordclass})}{\text{count}(\text{word}, \text{tag})}$$

This probability is inverted to calculate $P(\text{tag} \rightarrow \text{word})$ to be used in Equation 1.

B.2 Vertical Markovization

A common assumption in PCFG is the right-hand side of the grammar rules depends only on the left-hand side, which is the immediate parent of the nodes on the right-hand side. However, the probability of decompositions often depends on higher-order parents. For instance, Klein and Manning (2003) note that an NP with parent S (a subject NP) is 8.7 times more likely to expand as a pronoun compared to an NP with a parent VP.

The Stanford Unlexicalized PCFG Parser uses *parent annotation*, which is presented as a node relabeling transformation in Johnson (1998). The labels of the nodes in the parse tree are modified to contain information about their parents.

The Stanford Parser assumes that only the immediate previous ancestor of the node matters for a node's expansion (Klein and Manning, 2003). Hence, a set of selected nodes is annotated with their immediate parents, i.e., a vertical Markov order $v = 2$. For the example in Figure 8, "we" would be annotated as NP ^S (subject NP), and "last year" would be annotated as NP ^VP (temporal NP). This annotation helps distinguish subject

NPs from object NPs and temporal NPs but does not distinguish object NPs from temporal NPs.

The system excludes certain parent annotations based on a cutoff based on "frequency and the mutual information between the history and the expansions" (Klein and Manning, 2003).³¹

B.3 Horizontal Markovization

The vertical markovization introduces more nodes and, consequently, more rules in the grammar, amplifying the issues related to the sparsity of certain expansions in the training data. The sparsity problem is prominent in lexicalized PCFGs, and one of the proposed solutions is to Markovize the rules Collins (2003, p. 627).

Similarly, Klein and Manning (2003) uses horizontal markovization from the head-child outward, following the approach presented in Collins (1999). The n -ary rules are broken into multiple rules if n is greater than the horizontal Markov order h . For example, the output of the Stanford Parser for sentence 6 contains the rule $\text{VP} \rightarrow \text{VBD PP NP PP}$, as demonstrated in Figure 8. For $v = 1$ and $h = 1$, this rule is represented by the following rules:

1. $\langle \text{VP}: [\text{VBZ}] \rangle \rightarrow \text{VBZ}$
2. $\langle \text{VP}: [\text{VBZ}] \dots \text{PP} \rangle \rightarrow \langle \text{VP}: [\text{VBZ}] \rangle \text{PP}$,
3. $\langle \text{VP}: [\text{VBZ}] \dots \text{NP} \rangle \rightarrow \langle \text{VP}: [\text{VBZ}] \dots \text{PP} \rangle \text{NP}$
4. $\langle \text{VP}: [\text{VBZ}] \dots \text{PP} \rangle \rightarrow \langle \text{VP}: [\text{VBZ}] \dots \text{NP} \rangle \text{PP}$

where the square brackets indicate the head of the branch.

Klein and Manning (2003) use horizontal Markov order $h = 2$ but do not split intermediate states that occur less than ten times in the training set. For the example in Figure 8, $h = 2$ would result in $\langle \text{VP}: [\text{VBZ}] \dots \text{NP PP} \rangle$, but if this symbol occurs in less than ten instances in the dataset, it is collapsed to $\langle \text{VP}: [\text{VBZ}] \dots \text{PP} \rangle$, where $h = 1$.³²

B.4 Unary Annotations

Additionally, Klein and Manning (2003) marks every non-terminal node with one child with UNARY annotation, which they observed has increased parsing accuracy. Furthermore, the determiners and adverbs with no siblings were annotated with the labels UNARY-DT and UNARY-RB, respectively.

³¹Hence, they set the vertical Markov order $v \leq 2$.

³²Klein and Manning (2003) state that their system uses a horizontal Markov order of $h \leq 2$

Using these tags, their system can distinguish between the adverbs "as well" (not unary tagged) and "also" (unary tagged).

B.5 Tag Splitting

Moreover, [Klein and Manning \(2003\)](#) use parent annotation at the POS level. They note that this is useful in distinguishing the adverbs that commonly occur under ADVPs (also, now) from those that appear under VPs (n't, not) or NPs (only, just) ([Klein and Manning, 2003](#)). POS-level parent annotation also captures most of the grammatical distinctions between subordinating conjunctions (while, as, if), complementizers (that, for), and prepositions (of, in, from), all of which are assigned to the "IN" tag in Penn Treebank ([Klein and Manning, 2003](#); [Santorini, 1990](#)).

B.6 Function Word Lexicalization

The parser subcategorizes certain closed-class words by annotating their POS tags with the words. They note that this precisely captures the functionality of closed-class words and improves parser accuracy. [Klein and Manning \(2003\)](#) note that most subcategorizations they use already exist as other tag sets. For example, the Brown ([Francis, 1965](#)) and CLAWS (C5) ([Rayson and Garside, 1998](#)) tag sets have separate tags for different forms of the verbal auxiliaries ([Klein and Manning, 2003](#)).

B.7 Other Annotations

The Penn Treebank already contains certain functional tags, such as TMP-NP for temporal NPs, which the authors also include in their annotations. The same technique is also applied to other phrasal nodes. Additionally, certain NPs and VPs are annotated with their head tags to, for example, distinguish possessive NPs from other NPs using a POS-NP tag. Finally, base NPs (NPs that do not contain another NP in their span) are annotated as NP-B, the nodes that dominate a verbal node are marked DOMINATES-V, and NPs that contain another NP as their right-most descendant are labeled as RIGHT-REC-NP.

C Berkeley Neural Parser Details

C.1 Block Sparsity Constraint

In the second set of encoder blocks, block sparsity constraints ([Kitaev and Klein, 2018](#)) are implemented. Inside a transformer block, the block sparsity constraint replaces all instances where a vector $x = [x^{(c)}; x^{(p)}]$ is multiplied by a parameter matrix, i.e., $c = Wx$ with a *split form*: $c = [c^{(c)}; c^{(p)}] = [W^{(c)}x^{(c)}; W^{(p)}x^{(p)}]$. This means that attention is applied separately to content vectors ($x^{(c)}$) and position vectors ($x^{(p)}$) to calculate the keys and queries in self-attention sub-layers, i.e., $q \cdot k = q^{(c)} \cdot k^{(c)} + q^{(p)} \cdot k^{(p)}$. Similarly, the feed-forward sub-layers in the transformer blocks are also split into two independent portions.

C.2 Span Scores

$$v_{i,j} = [\vec{y}_j - \vec{y}_i; \overleftarrow{y}_{j+1} - \overleftarrow{y}_{i+1}]$$

, where $\vec{y}_t$ contains the even coordinates of y_t and $\overleftarrow{y}_t$ contains the odd coordinates. This notation is introduced in [Stern et al. \(2017\)](#), whose use of Bidirectional LSTM encoder makes the notation more intuitive, but [Kitaev and Klein \(2018\)](#) adopt the same notation in their paper despite their use of transformer architecture.

C.3 Conversion to Chomsky Normal Form

The model handles unary chains by collapsing the entry for each unary chain in the training set. n -ary trees are converted to binary trees by introducing a dummy label $\emptyset$, such that for all i and j $s(i, j, \emptyset) = 0$. This ensures that the dummy labels do not impact the tree scores.

D Preprocessing

Original	Edit
Gold Standard: "TOP"	removed
Gold Standard: Parentheses i.e., "(" and ")" tokens	replaced with -LRB- and -RRB-
Stanford Parser: "ROOT"	removed
All: Periods at the end	removed

Table 4

To ensure reliable comparisons of the outputted parse trees with the gold standard, I followed certain preprocessing steps to standardize the format of the parse trees. I removed the punctuation-capturing "TOP" constituent from the gold standard and the "ROOT" node from the Stanford PCFG Parser’s output to align their formats with the Berkeley Neural Parser. In this process, I noted that the punctuations at the end of sentences were attached to the "TOP" node in the gold standard, whereas in both Stanford and Berkeley parsers, they were descendants of "S". To avoid the higher error rates that would arise due to this difference in formatting convention, I removed the periods at the end of the sentences from the gold standard and the parser outputs, noting that all end-of-sentence periods were connected to "S" in the parser outputs and while all end-of-sentence periods in the gold standard were connected to the "TOP" node.

Moreover, I corrected the tags for parentheses in the gold standards, replacing '(' with the tags for right and left parenthesis from the Penn treebank. These were also the tags used by the Berkeley and Stanford parsers for the parentheses. Furthermore, I removed the word indexes from the gold standard to standardize the notation.

E Labeled Precision and Recall

ID	Len	LR	LP	CB	Tag Acc.
1	9	62.50	83.33	1	88.89
2	9	55.56	83.33	0	100.00
3	10	60.00	66.67	1	90.00
4	9	45.45	83.33	0	88.89
5	14	60.00	56.25	1	92.86
6	15	63.16	75.00	1	93.33
7	20	50.00	76.47	0	95.00
8	26	61.76	87.50	0	96.15
9	52	33.93	55.88	7	90.38
10a	17	38.89	63.64	3	100.00
10b	39	41.67	50.00	7	89.74
		48.35	66.86	21	92.73

Table 5: Berkeley Neural Parser EVALB Results. The bottom row shows the overall results. The parser achieves an overall Bracketing F-Score of 56.12.

ID	Len	LR	LP	CB	Tag Acc.
1	9	62.50	83.33	1	100.00
2	9	55.56	83.33	0	100.00
3	10	60.00	66.67	1	80.00
4	9	45.45	83.33	0	88.89
5	14	66.67	62.50	0	92.86
6	15	63.16	75.00	1	93.33
7	20	46.15	54.55	2	80.00
8	26	52.94	78.26	2	92.31
9	52	35.71	48.78	9	86.54
10a	17	38.89	70.00	2	94.12
10b	39	22.22	25.00	13	87.18
		44.63	57.75	31	89.09

Table 6: Stanford PCFG Parser EVALB Results. The bottom row shows the overall results. The parser achieves an overall Bracketing F-Score of 50.35. I also observed that the Stanford Parser has particularly low precision and recall for sentence 10b.

ID	Len	LR	LP	CB	Tag Acc.
1	9	100.00	100.00	0	88.89
2	9	100.00	100.00	0	100.00
3	10	100.00	100.00	0	70.00
4	9	100.00	100.00	0	100.00
5	14	93.75	93.75	1	100.00
6	15	100.00	100.00	0	100.00
7	20	76.47	59.09	1	85.00
8	26	79.17	82.61	1	96.15
9	52	76.47	63.41	5	88.46
10a	17	90.91	100.00	0	94.12
10b	39	40.00	37.50	10	89.74
		78.86	73.80	18	91.36

Table 7: Comparison of the Stanford PCFG Parser and Berkeley Neural Parser via EVALB Results. In this table, Stanford Parser is evaluated assuming the Berkeley Parser’s output to be the gold standard. Note that when the parsers change roles, the only change in the table is the switching of the precision and recall columns. It is observed that the outputs of the parsers match exactly for sentences 1, 2, 3, 4, and 6. I also note that the parser outputs are more aligned for shorter sentences, and significant deviations are observed for sentences that are longer than 20 tokens. The overall Bracketing F-Score is 76.24, which indicates a higher match between the parser outputs than either parser achieves with respect to the gold standard.

F VP Attachment Error - An Example from the Stanford Parser

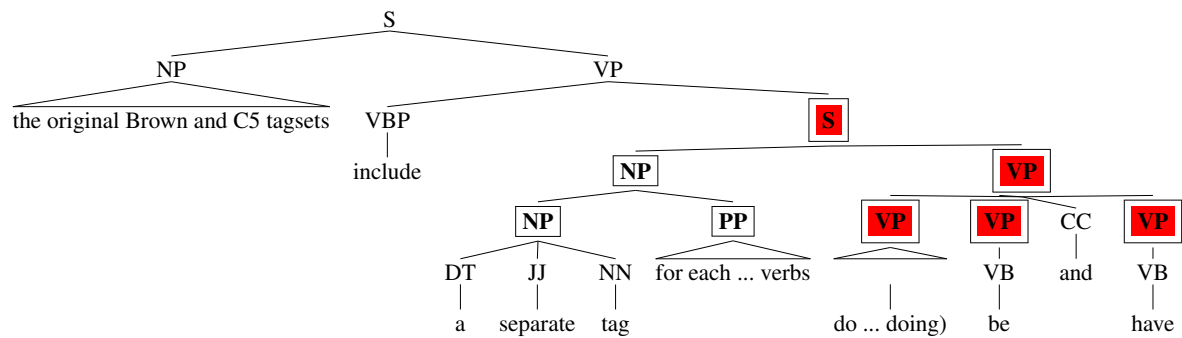


Figure 9: Stanford Parser Output for Sentence 10b

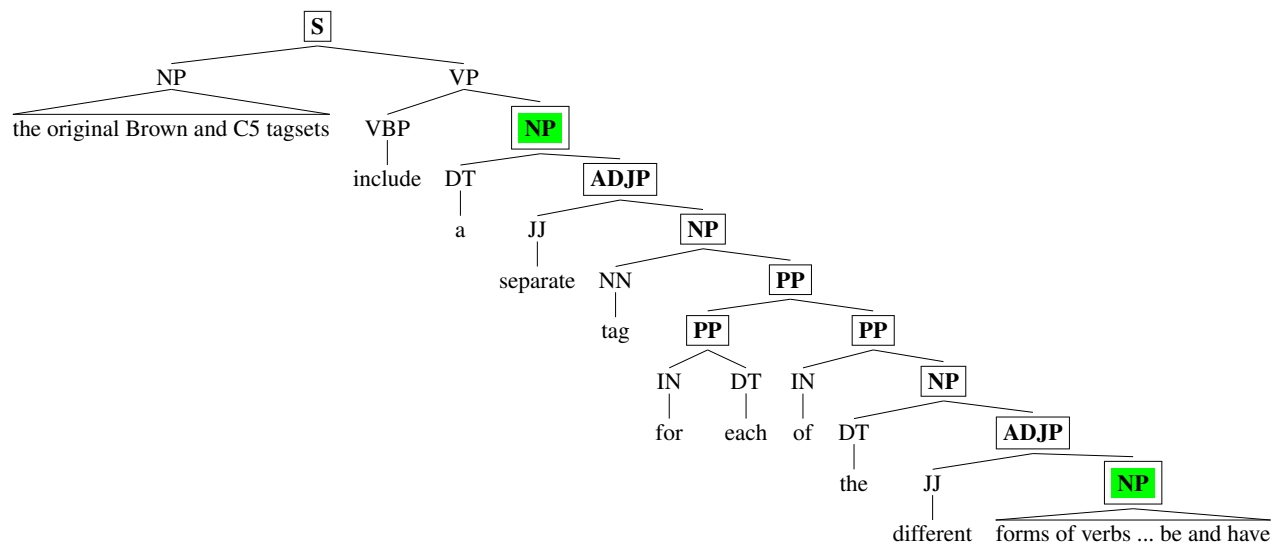


Figure 10: Gold Tree for Sentence 10b

G PP Attachment Error - An Example from the Berkeley Parser

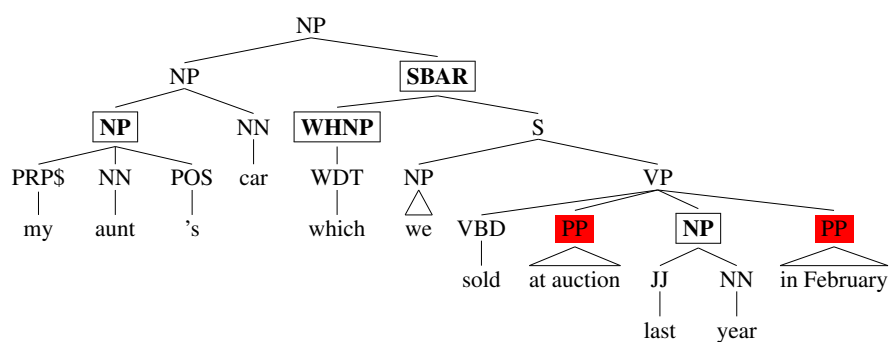


Figure 11: Berkeley Parser Output for Sentence 6

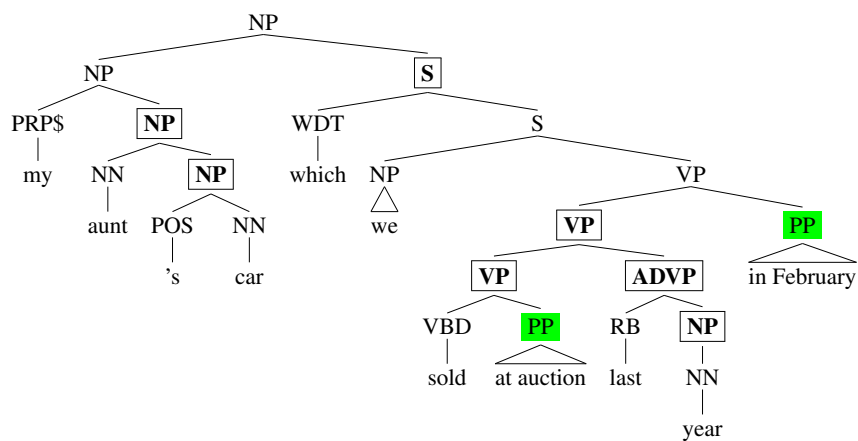


Figure 12: Gold Tree for Sentence 6

H NP Attachment Error - An Example from the Berkeley Parser

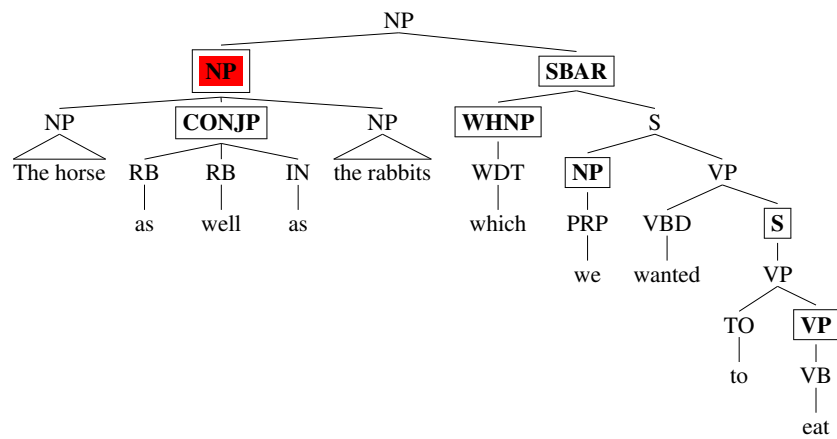


Figure 13: Berkeley Parser Output for Sentence 5

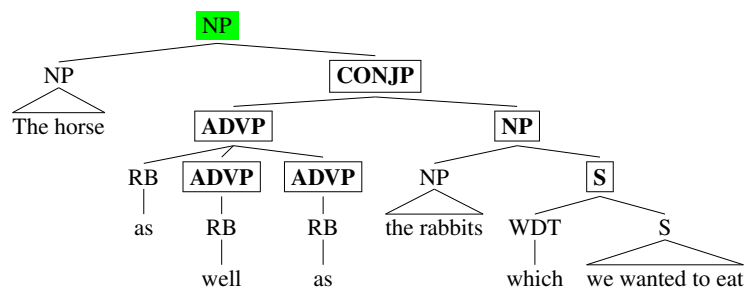


Figure 14: Gold Tree for Sentence 5

I Modifier Attachment Error - An Example from the Stanford Parser

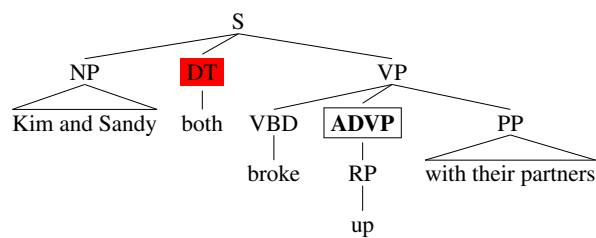


Figure 15: Stanford Parser Output for Sentence 4

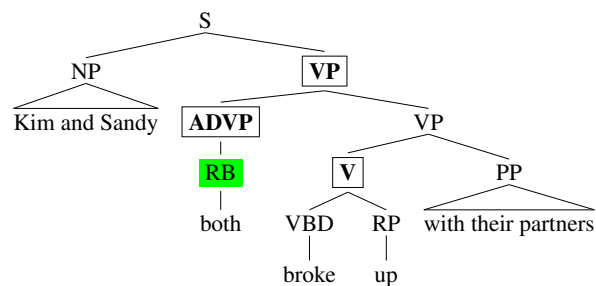


Figure 16: Gold Tree for Sentence 4

J Clause Attachment Error - An Example from the Berkeley Parser

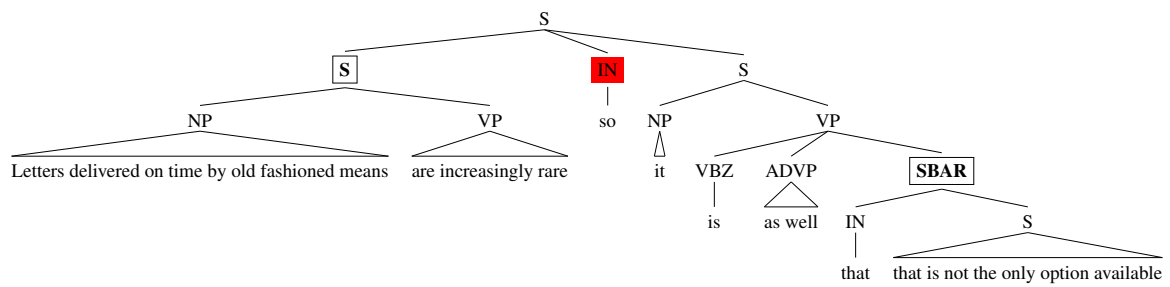


Figure 17: Berkeley Parser Output for Sentence 8

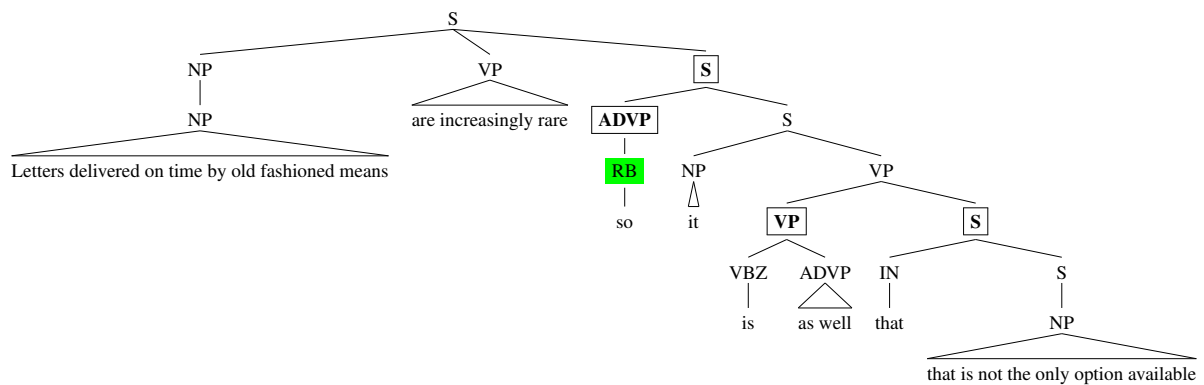


Figure 18: Gold Tree for Sentence 8

K POS Tags Statistics and Confusion Matrices

POS Tag	Precision	Recall
JJ	0.8235	1.0000
RBR	0.0000	NaN
NNPS	0.0000	NaN
DT	0.9412	1.0000
-LRB-	1.0000	1.0000
RP	1.0000	1.0000
VTB	1.0000	0.7500
VBD	0.8000	1.0000
PRP	1.0000	1.0000
”	1.0000	1.0000
:	1.0000	0.5000
JJR	NaN	0.0000
VBZ	1.0000	1.0000
CD	0.6667	1.0000
“	1.0000	1.0000
EX	1.0000	1.0000
VBP	1.0000	0.6000
-RRB-	1.0000	1.0000
NNP	0.7857	1.0000
JJS	0.0000	NaN
VB	0.7143	0.8333
NN	1.0000	0.8000
MD	1.0000	1.0000
HYPH	1.0000	1.0000
PRP\$	1.0000	1.0000
CC	1.0000	1.0000
TO	1.0000	1.0000
RBS	NaN	0.0000
SYM	0.0000	NaN
WDT	1.0000	1.0000
POS	1.0000	1.0000
IN	0.8571	1.0000
NNS	1.0000	0.9474
VBG	0.6667	1.0000
UH	0.3333	0.3333
RB	1.0000	0.6111
,	1.0000	1.0000
FW	0.0000	NaN

Table 8: Stanford PCFG Parser POS Tag Statistics.^a

^aNaN indicates division by 0.

Label	Precision	Recall
JJ	0.9333	1.0000
RBR	0.0000	NaN
NNPS	NaN	NaN
DT	0.9412	1.0000
-LRB-	1.0000	1.0000
RP	1.0000	0.6667
VTB	1.0000	0.7500
VBD	0.8889	1.0000
PRP	1.0000	1.0000
”	1.0000	1.0000
:	1.0000	1.0000
JJR	NaN	0.0000
VBZ	1.0000	1.0000
CD	0.6667	1.0000
“	1.0000	1.0000
EX	0.5000	1.0000
VBP	1.0000	1.0000
-RRB-	1.0000	1.0000
NNP	0.8333	0.9091
JJS	NaN	NaN
VB	0.8571	1.0000
NN	1.0000	0.8667
MD	0.5000	1.0000
HYPH	1.0000	1.0000
PRP\$	1.0000	1.0000
CC	1.0000	1.0000
TO	1.0000	1.0000
RBS	1.0000	1.0000
SYM	NaN	NaN
WDT	1.0000	1.0000
POS	1.0000	1.0000
IN	0.8571	1.0000
NNS	1.0000	1.0000
VBG	0.6667	1.0000
UH	0.6667	0.6667
RB	1.0000	0.6667
,	1.0000	1.0000
FW	NaN	NaN

Table 9: Berkeley Neural Parser POS Tag Statistics.^a

^aNaN indicates division by 0.

[illegible]

Figure 19: Berkeley Neural Parser POS Tags Confusion Matrix. Columns are the parser outputs. Rows are the true labels from the Gold Standard. The cell (i, j) represents how many times the true label i is classified as j by the parser.

[illegible]

Figure 20: Stanford PCFG Parser POS Tags Confusion Matrix. Columns are the parser outputs. Rows are the true labels from the Gold Standard. The cell (i, j) represents how many times the true label i is classified as j by the parser.